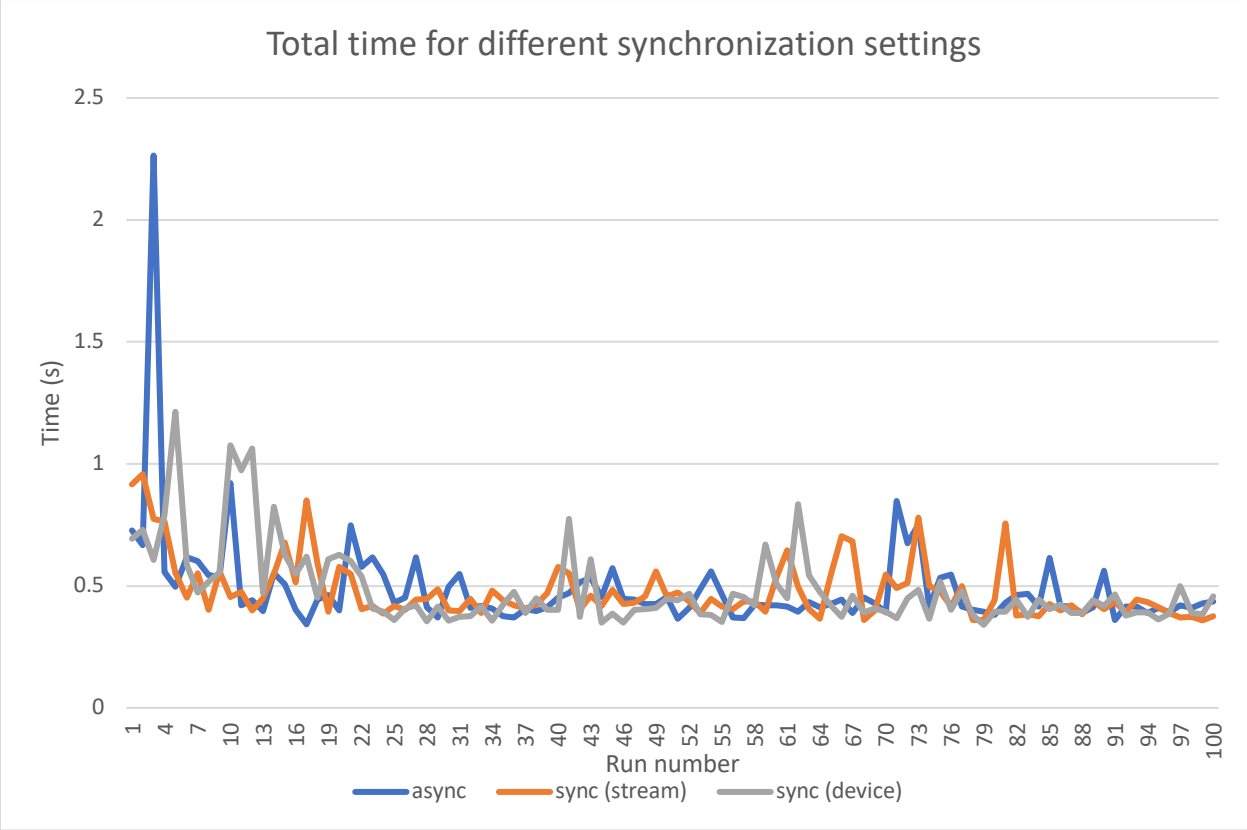
Experiments

Initial Performance and Synchronous/Asynchronous calls

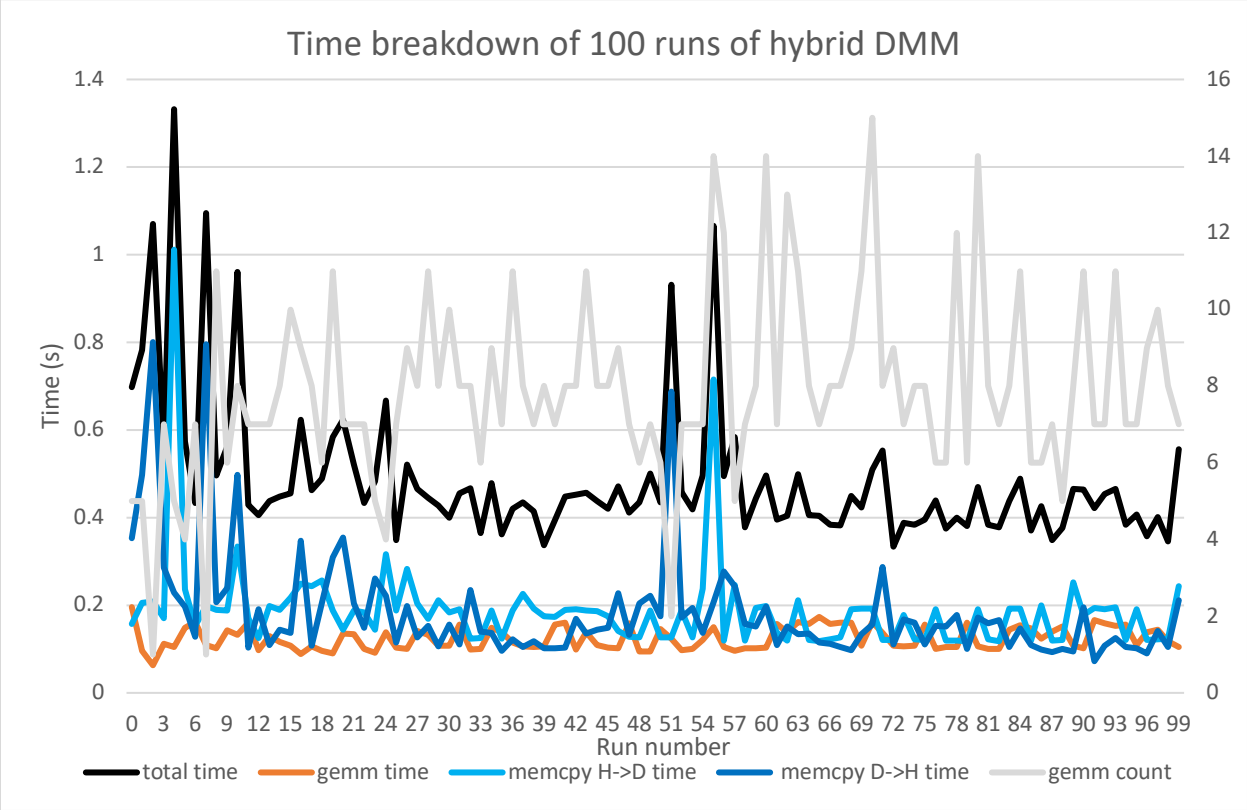
We originally ran 100 runs of the hybrid algorithm one after another and recorded the time it took. We found that the total time was quite inconsistent, and the first few runs had much worse performance (the warm-up).

We tested both using synchronous and asynchronous CUDA calls and found not much of a difference in the timing.



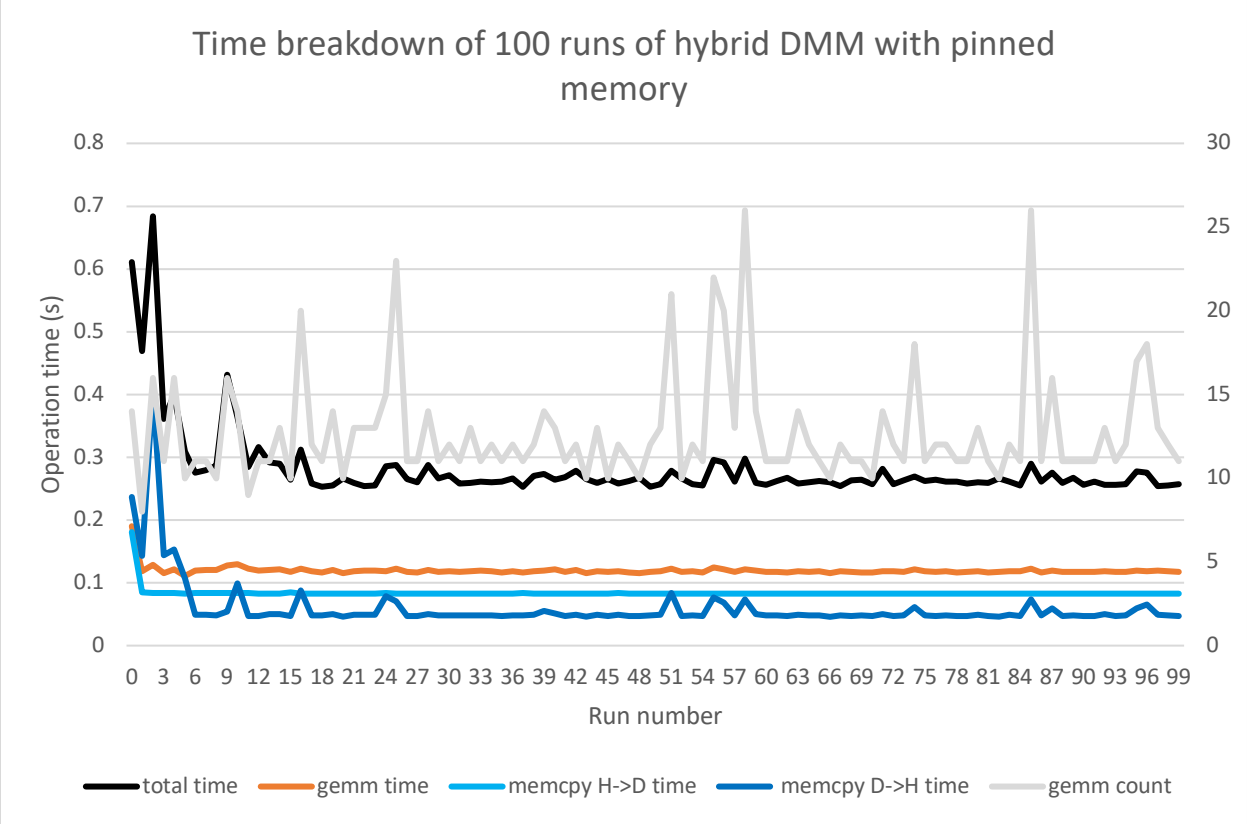
Analysis

Breaking down the runs, we found that the vast majority of the time was spent during the memcpy operation between the host and device (both ways) - making up around 85% of the total time. This was also the biggest source of inconsistency in times across runs. Memcpy time was pretty inconsistent with respect to size, while gemm time correlated more strongly with respect to size.

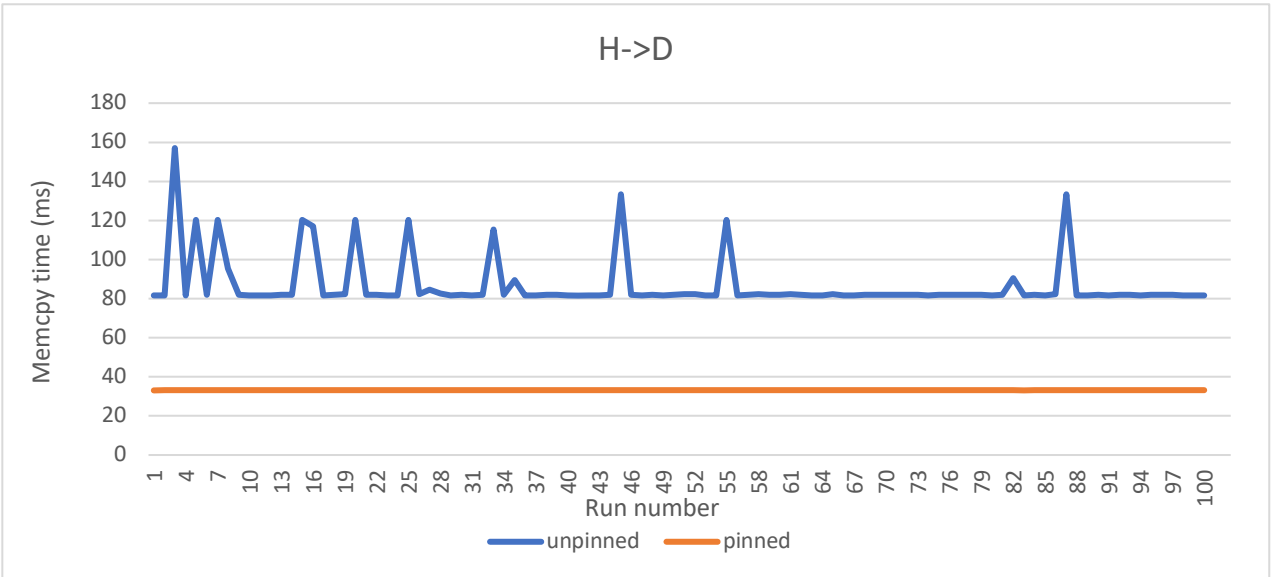


Pinned Memory

The experiment we tested that brought the biggest change was using CUDA's pinned memory. Pinned memory brought a significant speed boost to the algorithm and also greatly reduced the inconsistencies in memcpy operations. Interestingly, it also seemed to make GEMM times more consistent.



Running a dummy kernel that just did a memcpy and a memory write, we saw a similar pattern, with memcpys being more time consuming and less consistent from unpinned memory than pinned memory.

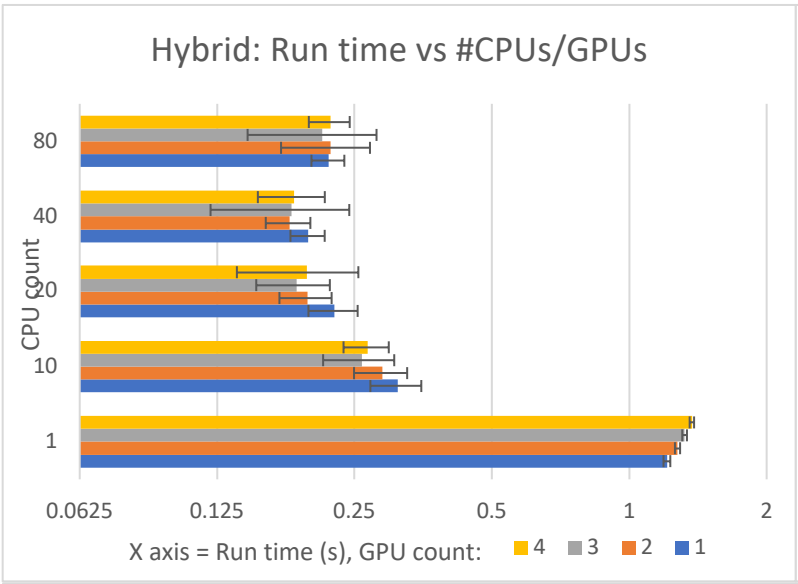


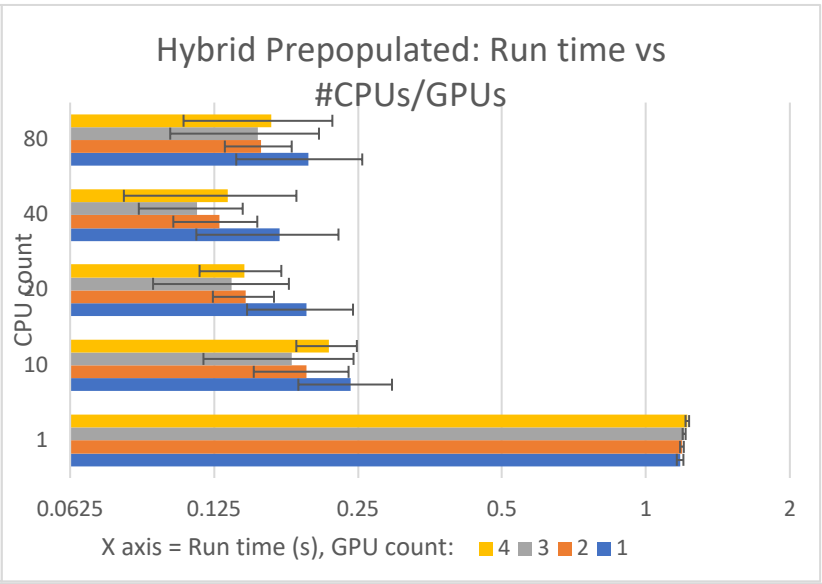
Challenges with Pinned Memory

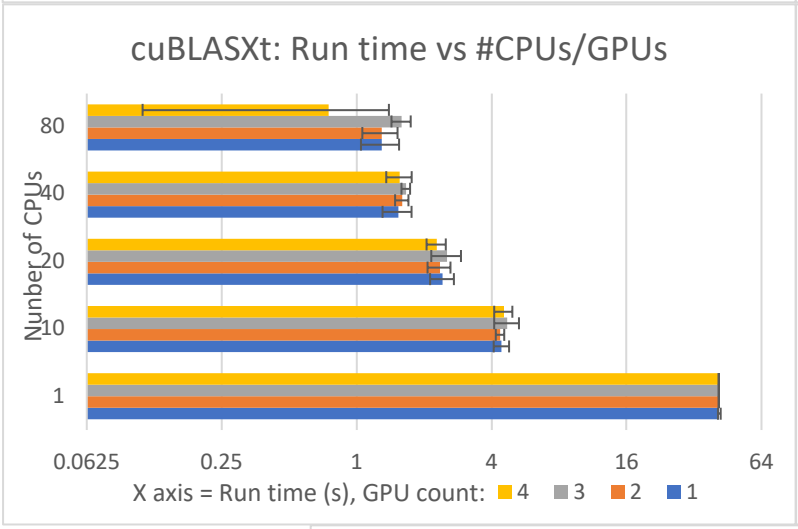
We found that pinned memory brought an additional challenge with it - the need to unpin memory eventually. Unpinning memory is an expensive operation best done in batch. Additionally, the existing test program would use a dynamically allocated and garbage collected array as the destination matrix which resulted in issues with the garbage collector clashing with double-pinning memory (before, we unpinned memory manually) on systems with multiple GPUs. This pinned memory allocator remains a further experiment to explore. For the purpose of testing, we unpinned memory at the end and didn't count it in the times, which is in my opinion an unfair test – a pinned memory allocator would be able to provide a fairer comparison.

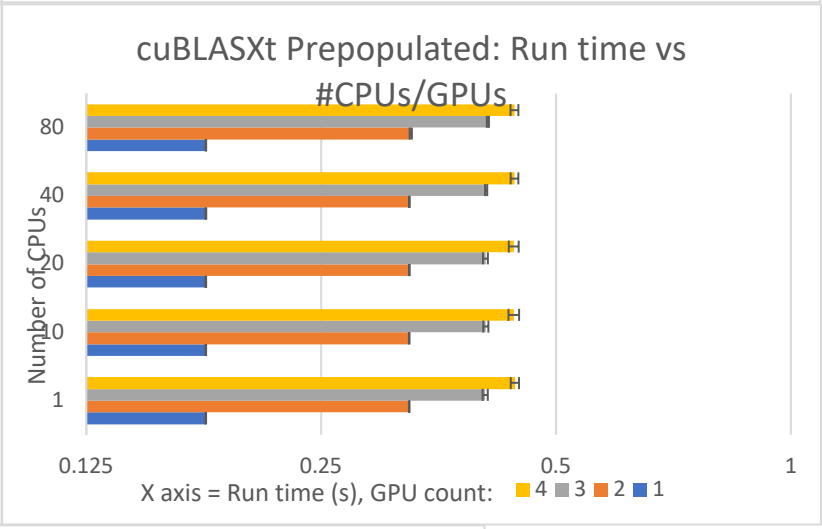
A100 runs

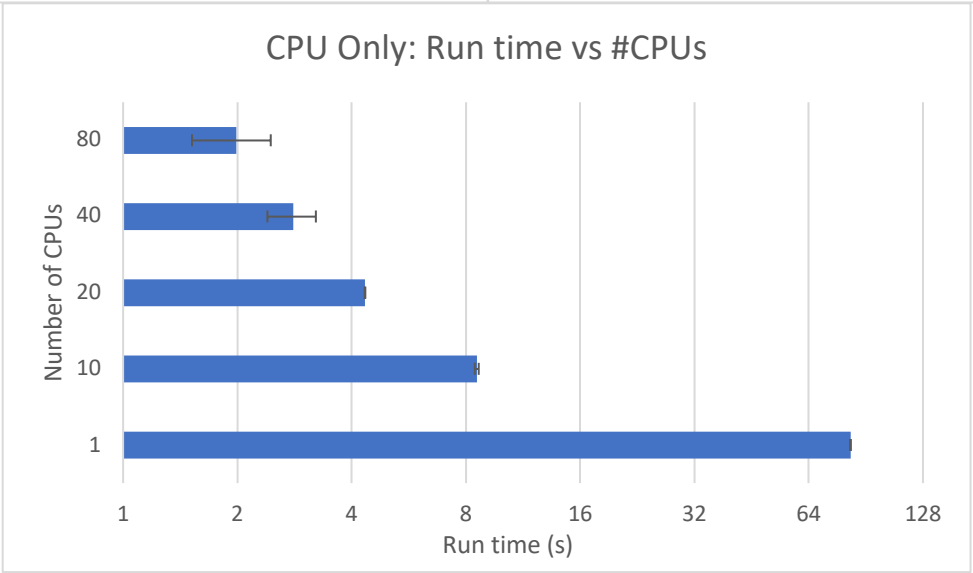
We ran several experiments on a 4x NVIDIA A100 GPU machine, testing different parameters: CPU count, GPU count, whether the input matrix was prepopulated. In addition to the hybrid system, we also tested using cuBLASXt and a CPU-only approach. These were run with pinned memory enabled everywhere.











Findings

The CPU-only system scaled the best but performed the worst overall.

cuBLASXt did not take advantage of being given more GPUs on our input of 10kx10k matrices. When prepopulated, it seemed to only use the GPU and not scale at all.

Both Hybrid and cuBLASXt scaled with CPU count, with diminishing returns. Hybrid 80-core generally performed worse than Hybrid 40-core. Hybrid also seemed to scale a bit when given more GPUs, up to 3. There is probably more work that can be done to make Hybrid scale better for multiple GPUs.

Associated Data

For a deeper dive into the data involved in these graphs, check out the following spreadsheets, which contain all the graphs in this document and more, and have some comments explaining each page:

cublasxt-benchmark.xlsx: Contains original runs of cuBLASXt, used to determine the optimal CPU ratio parameter of around 0.5-0.6. For our runs afterward, we used 0.5.

hybrid-timings.xlsx: The meat of the report's data, contains all the experiments I performed in understanding and trying to optimize the performance of the Hybrid system for the DMM benchmark. A few other experiments that didn't produce particularly interesting results are described there too. The pages in this spreadsheet are listed in chronological order, showing experiments and improvements in the order that they were made.

a100-run-graphs.xlsx: Contains runs of code with pinned memory optimizations run on the A100 machine. Good for comparisons.